AIX-MARSEILLE UNIVERSITÉ
MASTER 2 SID
Apprentissage automatique pour la science des données
Année universitaire 2026-2027

TP N°1 : MANIPULATION DE LA BASE NOSQL NEO4J

PAR MURIEL VISANI

Ce TP a pour objectif de vous montrer quelques exemples de commandes *Cypher* sous Neo4j.

Usage de l'IA :

L'objectif de ce TP est de vous faire prendre un main un nouveau type de base de données, et un nouveau langage de requêtage, et que vous compreniez / maîtrisiez vraiment ce que vous faites. Pour chaque question qui nécessite une nouvelle notion, je vous donne le lien vers la documentation nécessaire. Ce TP n'est pas noté, et il n'a que peu d'intérêt si vous utilisez l'IA générative pour créer vos requêtes.

Donc, n'utilisez pas l'IA générative pour ce TP !!!

I. Exploration de la base « movies »

Nous allons d'abord travailler avec une base de films. Par exemple, chaque personne dans cette base peut être acteur, réalisateur ou producteur d'un ou plusieurs films.

a. Connexion au serveur de démo en ligne de Neo4j

Rendez-vous sur la page :
https://demo.neo4jlabs.com:7473/browser/?dbms=neo4j://movies@demo.neo4jlabs.com&db=movies

Connectez-vous à cette instance en remplissant les champs « Protocol » avec « neo4j+s// » et « Password » avec « movies ».

b. Exploration de la base « movies »

Vous arrivez normalement sur une page appelée « Database Information ».

1. Combien de nœuds comporte cette base ?

2. Il existe deux types de nœuds dans cette base : quels sont-ils ?

3. Combien de types de relations existent dans cette base ? Quelles sont ces relations ? Pour chaque type de relation, dites entre quels types de nœuds elle opère selon vous.

4. Combien de propriétés existent dans cette base ? En cliquant sur les propriétés, vous pourrez voir sur quel type d'entité chaque propriété opère, et observez bien le code Cypher qui permet de retourner le résultat. Pour les propriétés « born », « name », « released », « votes », « roles » et « summary » dites sur quels type d'entité (nœud ou arête *a.k.a.* relationship) elles opèrent. Est-ce que cela vous semble logique ?

5. Exécuter la requête suivante permettant de récupérer l'ensemble du graphe (nom, source et destination) :

   CALL db.schema.visualization()

II. Requêtes sur la base « movies »

Dans Cypher, la clause qui permet de faire des requêtes est la clause « MATCH » suivie de « RETURN » (c'est à voir comme l'équivalent de SELECT en SQL).

Vous trouverez plus d'informations sur le MATCH dans cette page : https://neo4j.com/docs/cypher-manual/current/clauses/match/

Ecrire et exécutez les requêtes Cypher permettant de répondre aux questions suivantes :

1. Nombre total de nœuds dans le graphe de la base de données 'Movies'.
2. Nombre total de films dans le graphe de la base de données 'Movies'.
3. Liste des titres de films dans la base « movies ».
4. Toutes les propriétés du nœud « Person » dont la propriété « name » est « Jack Nicholson » ?
5. Quels types de relations ont la propriété « summary » ?
6. Pour chaque relation pour laquelle la propriété « summary » est remplie : donnez le nom de la personne ayant « reviewé » le film, le titre du film, le contenu de « summary », et la note ('rating') donnée par cette personne dans sa review.
7. Donnez tous les « summary » du film « "The Da Vinci Code" ».
8. Tous les titres de films 'reviewed' par l'utilisateur 'Jessica Thompson', avec le 'rating' et le summary associé, ordonné par 'rating' décroissant (le 'rating' sera nommé 'note'.
9. Tous les titres de films qui ne sont pas 'reviewed' par l'utilisateur 'Jessica Thompson'. **Indice :** regarder la page https://neo4j.com/docs/cyphermanual/current/clauses/where/#filter-on-patterns-using-not
10. Les titres et dates de sortie des films dont le titre contient 'Matrix' (quelle que soit la casse : majuscules . **Indice** : regarder la page https://neo4j.com/docs/cypher-manual/current/clauses/where/#matchstring-contains
11. Les titres des films dont le titre commence par 'The Matrix Re', sans tenir compte de la casse. **Indice** : regarder la page https://neo4j.com/docs/cypher-manual/current/clauses/where/#matchingusing-regular-expressions
12. Les titres de films qui sont notés par l'utilisateur 'Jessica Thompson' (dans l'ordre alphabétique du titre du film) et la liste des acteurs du film. Même si le film n'a aucun acteur, on veut quand même afficher son titre. **Indice :** regarder la page https://neo4j.com/docs/cypher-manual/current/functions/aggregating/#functions-collect

13. Le nombre de films par acteur, par ordre décroissant de nombre de films joués. **Indice :** regarder la page https://neo4j.com/docs/cyphermanual/current/subqueries/count/

III. Création et manipulations d'une nouvelle base

Créez un compte gratuit sur neo4j (AuraDB) : https://neo4j.com/ en cliquant sur 'Get Started for Free', entrant une adresse email puis continuez le processus d'enregistrement jusqu'au bout, puis cliquez sur « Create a free instance ». Cela peut prendre quelques minutes.

Vous allez créer le graphe d'une université. Pour cela, vous allez créer les entités une à une, puis leurs relations.

Les nœuds à créer sont les suivants : Student, Professor, Course, Department, Room.

Chaque département et chaque professeur a une seule propriété : 'name'.

Chaque cours a trois propriétés : 'code', 'title', 'credits'.

Chaque salle a deux propriétés : 'name', 'capacity'.

Chaque étudiant a deux propriétés : 'name' et 'year'.

Les relations (arêtes) sont les suivantes :

- (Student)-[:ENROLLED_IN {grade, year}]->(Course)
- (Student)-[:MEMBER_OF]->(Department)
- (Student)-[:FRIEND_OF]->(Student)
- (Professor)-[:TEACHES]->(Course)
- (Professor)-[:WORKS_IN]->(Department)
- (Professor)-[:HEADS]->(Department)
- (Course)-[:OFFERED_BY]->(Department)
- (Course)-[:PREREQUISITE_OF]->(Course)
- (Course)-[:HELD_IN]->(Room)

1. Créez ce graphe, en utilisant bien les noms exacts des nœuds et des relations donnés ci-dessus. Pour cela, créez :
    - Deux départements : Mathématiques et Informatique
    - Trois professeurs : Claire Martin, Paul Durand et Sophie Leroy

- Quatre cours, deux salles et trois étudiants. On considère que chaque cours est mené dans une seule salle. Chaque étudiant suit au moins un cours, et un étudiant suit au moins deux cours.

- Pensez à bien peupler tous les types de relations.

Indice : Regardez la page https://neo4j.com/docs/cypher-manual/current/clauses/create/

2. Avec CALL db.schema.visualization() vérifiez que votre schéma de base est le bon.

3. Rajoutez un attribut DOB (date of birth) aux étudiants déjà créés.

4. Recherchez le professeur Paul Durand et rajoutez-lui la DOB du 01/01/1980.

5. Créez un autre département 'Biologie', et rajoutez une relation CO_HEADS entre Paul Durand et ce nouveau département. Pour éviter de créer deux départements Biologie si on relance la requête, utilisez MERGE au lieu de CREATE.

6. Ecrire et exécuter la requête permettant de calculer le nombre de cours suivis par étudiant, et ajouter un attribut courseCount aux étudiants. Vérifier la mise à jour en affichant tous les étudiants et leurs propriétés.

7. Selon vous, quel est le choix de base de données le plus pertinent (base de données relationnelle ou Neo4J) :
   - pour la gestion courante d'une université (inscriptions, affectation des cours aux professeurs…) ?
   - pour l'analyse du graphe : analyse des prérequis de cours, analyse des réseaux d'amis d'étudiants

8. **Question bonus** : trouver cinq questions qui nécessitent des requêtes assez complexes, et soumettez-les à vos voisins.